

Universidad
Manuela Beltrán
Acreditada en Alta Calidad Multicampus

Análisis estático de seguridad (SAST) con gosec

Informe de la Guía 7: instalación, hallazgos y experiencia de uso

Cobos, Dilan Steven

Rodríguez Ángel, Santiago

Alarcón Ospina, Víctor Alfonso

Universidad Manuela Beltrán

Ingeniería de Software

Facultad de Ingeniería

Desarrollo de Software Seguro

Bogotá, Colombia

2026

Tabla de contenido

Introducción 5
1. Marco conceptual 5
1.1 Análisis estático de seguridad 5
1.2 ¿Qué es gosec? 6
1.3 Severidad y confianza 6
2. Configuración del entorno 7
2.1 Equipo y requisitos 7
2.2 Instalación 7
2.3 Objetivos analizados 8
3. Metodología 8
3.1 Procedimiento 8
3.2 Criterios de análisis 8
3.3 Reproducibilidad 9
4. Resultados sobre GoVWA 9
4.1 Panorama general 9
4.2 Hallazgos por regla 10
4.3 Hallazgos destacados 12
4.3.1 Inyección SQL (G701 y G201) 12
4.3.2 Contraseñas protegidas con MD5 (G401 y G501) 12
4.3.3 HTML sin escapar (G203) 13
4.4 Otros hallazgos 13
5. Código propio: antes y después 14
5.1 Diseño del ejemplo 14
5.2 Falso positivo y supresión justificada 15
6. Límites de la herramienta 15
6.1 Fallos de lógica: el caso del IDOR 15
6.2 Secretos en archivos de configuración 16
6.3 Credenciales en el código: la regla G101 16
6.4 Falsos positivos y confianza baja 16
7. Experiencia de uso 17
8. Recomendaciones 18
Conclusiones 18
Referencias 20
Anexo A. Reproducción de los resultados 21

Lista de tablas

Tabla 1 Características de gosec v2.30.0 6
Tabla 2 Entorno de las ejecuciones 7
Tabla 3 Hallazgos de gosec sobre GoVWA por severidad y confianza 10
Tabla 4 Reglas de gosec activadas sobre GoVWA 10
Tabla 5 Riesgo y mitigación recomendada por regla 11
Tabla 6 Resultados de gosec sobre el código propio antes y después de corregir 14
Tabla 7 Errores del código propio y corrección aplicada 14
Tabla 8 Variables con credenciales marcadas por la regla G101 16
Tabla 9 Valoración de la experiencia de uso de gosec 17

Lista de figuras

Figura 1 Informe HTML de gosec sobre GoVWA 9
Figura 2 Hallazgos de gosec sobre GoVWA por regla y severidad 11

Introducción

Las pruebas estáticas de seguridad de aplicaciones (SAST, por las siglas en inglés de Static Application Security Testing) revisan el código fuente sin ejecutar la aplicación y buscan patrones asociados a vulnerabilidades conocidas (OWASP Foundation, s. f.). Su principal ventaja es que permiten detectar problemas desde que el código se escribe, cuando corregirlos es más sencillo.

Este informe documenta la Guía 7 de la asignatura Desarrollo de Software Seguro, cuyo tema asignado al grupo es gosec, un analizador SAST para el lenguaje Go mantenido por el proyecto securego (securego, s. f.). El objetivo fue instalar y configurar la herramienta, analizar una aplicación vulnerable y código propio, documentar las vulnerabilidades halladas con su severidad y sus medidas de mitigación, y valorar la experiencia de uso en cuanto a instalación, facilidad, resultados e interfaz.

El documento se organiza en ocho secciones. La sección 1 presenta el marco conceptual; la 2, la configuración del entorno; la 3, la metodología; la 4, los resultados sobre la aplicación vulnerable; la 5, el ciclo de detección y corrección con código propio; la 6, los límites de la herramienta; la 7, la experiencia de uso, y la 8, las recomendaciones. Todas las cifras provienen de ejecuciones reales de gosec conservadas en la carpeta de resultados del trabajo, por lo que pueden repetirse con los scripts que se describen en el Anexo A.

1. Marco conceptual

1.1 Análisis estático de seguridad

El análisis estático examina el código sin ejecutarlo, de forma comparable a revisar los planos de una casa antes de construirla. Su alcance es el código que recibe: no observa el comportamiento de la aplicación en funcionamiento, ni los componentes de terceros, ni la

configuración del servidor. Por esa razón se complementa con las pruebas dinámicas (DAST), que atacan la aplicación mientras se ejecuta, y con el análisis de composición de software (SCA), que revisa las dependencias.

1.2 ¿Qué es gosec?

gosec es un analizador de seguridad de código abierto para el lenguaje Go, publicado bajo licencia Apache 2.0 (securego, s. f.). Examina el árbol sintáctico (AST) y la representación intermedia SSA del programa y aplica un conjunto de reglas, cada una con un identificador propio (de G101 a G710) y asociada a una debilidad del catálogo Common Weakness Enumeration, CWE (MITRE, s. f.). Las reglas de la serie G70x añaden análisis de flujo de datos, que sigue un dato desde la entrada del usuario hasta una función peligrosa. La Tabla 1 resume las características de la versión utilizada.

Tabla 1
Características de gosec v2.30.0

Característica	Descripción
Lenguaje analizado	Go (solo archivos .go)
Licencia	Apache 2.0
Reglas	62, de G101 a G710, cada una con su CWE
Técnicas	Árbol sintáctico (AST), representación SSA y análisis de flujo de datos (G701 a G710)
Filtros	Por severidad y confianza, e inclusión o exclusión de reglas
Formatos de salida	text, json, yaml, csv, junit-xml, html, sonarqube, golint y sarif
Integración continua	GitHub Action oficial (securego/gosec)
Requisito	Go 1.25 o superior

Nota. Elaboración propia a partir de la documentación del proyecto (securego, s. f.) y de la ejecución de la herramienta.

1.3 Severidad y confianza

Cada hallazgo de gosec se califica en dos ejes independientes. La severidad (alta, media o baja) indica qué tan grave sería el problema si fuera real. La confianza (alta, media o baja) indica qué tan seguro está gosec de que el hallazgo corresponde a un problema real. Un hallazgo de

severidad media y confianza baja merece una revisión humana antes de actuar, mientras que uno de severidad alta y confianza alta debe atenderse primero. Esta distinción guía la priorización de las secciones 4 y 8.

2. Configuración del entorno

2.1 Equipo y requisitos

La instalación y los análisis se realizaron en un equipo con Windows 11 y Go 1.27.2, sin Docker, sin cuentas en servicios externos y sin permisos de administrador. La Tabla 2 reúne los datos del entorno exacto, registrados por el script de escaneo para que el análisis sea repetible.

Tabla 2
Entorno de las ejecuciones

Elemento	Valor
Sistema operativo	Windows 11 Home
Go	1.27.2 (windows/amd64)
gosec	v2.30.0
Aplicación vulnerable	GoVWA, commit 4058f79
Código analizado	20 archivos y 1.569 líneas
Fecha de las ejecuciones	23 de octubre de 2026
Requisitos adicionales	Ninguno

Nota. Datos del archivo resultados/entorno.txt y de la ejecución de gosec.

2.2 Instalación

La instalación consiste en un solo comando que descarga y compila la herramienta desde su código fuente:

```
go install github.com/securego/gosec/v2/cmd/gosec@latest
```

La compilación tardó aproximadamente 1 minuto y 50 segundos y dejó el ejecutable, de unos 97 MB, en la carpeta de binarios de Go. No fue necesario instalar nada más. Un detalle práctico es que, al instalar desde el código fuente, el comando gosec -version responde «dev» en lugar del número de versión. La versión real se consulta con el siguiente comando, que reportó la v2.30.0:

```
go version -m gosec.exe
```

Por eso conviene registrar la versión real en cualquier informe de análisis; también explica que el resumen de los resultados de texto muestre «Gosec: dev».

2.3 Objetivos analizados

Se analizaron dos objetivos. El primero fue GoVWA (Go Vulnerable Web Application), una aplicación web escrita en Go con vulnerabilidades introducidas a propósito para aprender a detectarlas (0c34, s. f.); se fijó el commit 4058f79 para que el análisis sea repetible. El segundo fue código propio: un módulo pequeño con los errores más comunes de Go y su versión corregida, que permite medir el efecto de aplicar las mitigaciones (sección 5).

3. Metodología

3.1 Procedimiento

En cada objetivo se ejecutó gosec en varios formatos de salida y con la opción -no-fail, que evita que el código de salida distinto de cero (el que gosec devuelve cuando hay hallazgos) interrumpa los scripts. Los comandos principales, simplificados, fueron los siguientes:

```
gosec -no-fail -fmt=text  -out=resultados/govwa.txt   ./objetivos/govwa/...
gosec -no-fail -fmt=json  -out=resultados/govwa.json  ./objetivos/govwa/...
gosec -no-fail -fmt=sarif -out=resultados/govwa.sarif ./objetivos/govwa/...
gosec -no-fail -fmt=html  -out=resultados/govwa.html  ./objetivos/govwa/...
gosec -no-fail -severity=high -confidence=high -fmt=text \
      -out=resultados/govwa_solo_graves.txt ./objetivos/govwa/...
```

Para el código propio se añadió la opción -track-suppressions, que registra los hallazgos suprimidos con comentarios #nosec. Los archivos JSON se procesaron con scripts en Python que resumen los hallazgos por severidad, confianza, regla y archivo.

3.2 Criterios de análisis

Los hallazgos se clasificaron por severidad y confianza. Se revisó manualmente el código de los hallazgos de severidad alta y de los de confianza baja, y cada hallazgo destacado se contrastó con la versión segura equivalente cuando el propio proyecto la incluía. En el código propio se midió el resultado antes y después de corregir.

3.3 Reproducibilidad

Los comandos, el commit del objetivo y las versiones de las herramientas quedaron registrados. Las rutas absolutas del equipo se eliminaron de los resultados para poder compartirlos. El Anexo A reúne los pasos para repetir el análisis.

4. Resultados sobre GoVWA

4.1 Panorama general

Sobre GoVWA, gosec analizó 20 archivos con 1.569 líneas y reportó 35 hallazgos: 1 de severidad alta, 16 de severidad media y 18 de severidad baja. Por confianza, 28 son de confianza alta y 7 de confianza baja. El único hallazgo de severidad alta es una inyección SQL con confianza alta, por lo que es el primero que debe atenderse. La Figura 1 muestra cómo presenta gosec sus hallazgos en el informe HTML, con filtros por severidad y confianza, y la Tabla 3 los cruza por ambos ejes.

Figura 1
Informe HTML de gosec sobre GoVWA



Nota. Captura real del informe generado con -fmt=html: filtros, resumen y los hallazgos G701 (inyección SQL) y G124 (cookie sin atributos seguros).

Tabla 3
Hallazgos de gosec sobre GoVWA por severidad y confianza

Severidad	Confianza alta	Confianza baja	Total
Alta	1	0	1
Media	9	7	16
Baja	18	0	18
Total	28	7	35

Nota. Elaboración propia a partir de resultados/govwa.json.

4.2 Hallazgos por regla

Los 35 hallazgos corresponden a nueve reglas (Tabla 4 y Figura 2). Las de mayor impacto son la inyección SQL (G701 y G201), la criptografía débil (G401 y G501) y el HTML sin escapar (G203); las de severidad baja son las más numerosas, pero casi todas se refieren a errores sin manejar y a escritura de datos externos en los registros. La Tabla 5 resume el riesgo de cada regla y la mitigación recomendada.

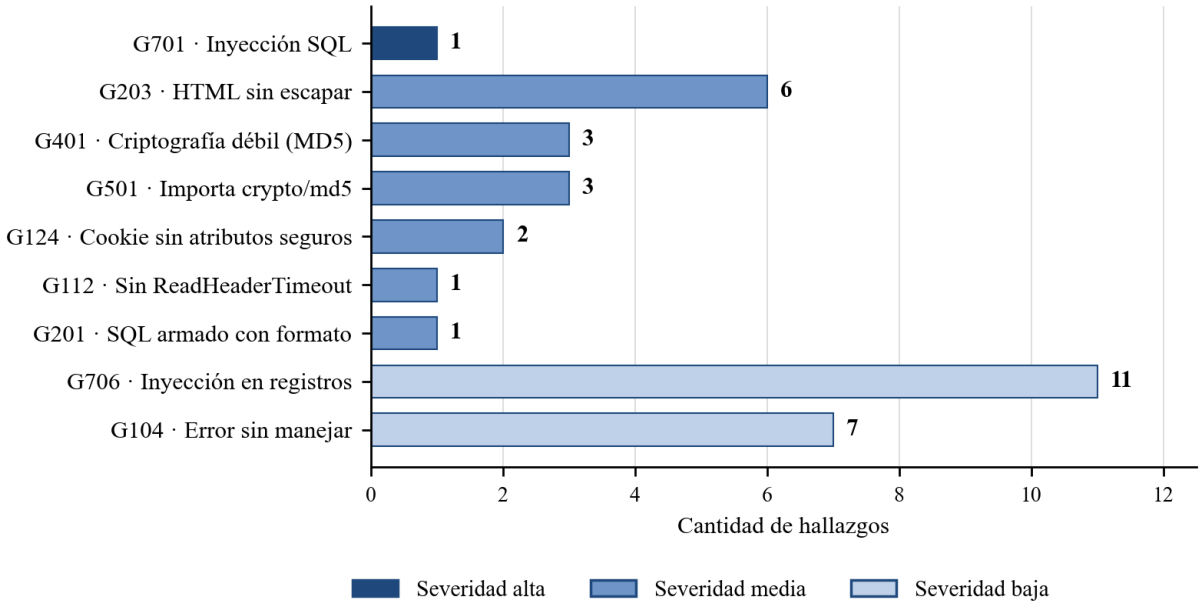
Tabla 4
Reglas de gosec activadas sobre GoVWA

Regla	CWE	Severidad	Confianza	N.°	Hallazgo
G701	CWE-89	Alta	Alta	1	Inyección SQL
G203	CWE-79	Media	Baja	6	HTML sin escapar
G401	CWE-328	Media	Alta	3	Criptografía débil (MD5)
G501	CWE-327	Media	Alta	3	Importa crypto/md5
G124	CWE-614	Media	Alta	2	Cookie sin atributos seguros
G112	CWE-400	Media	Baja	1	Sin ReadHeaderTimeout
G201	CWE-89	Media	Alta	1	SQL armado con formato
G706	CWE-117	Baja	Alta	11	Inyección en registros
G104	CWE-703	Baja	Alta	7	Error sin manejar

Nota. Elaboración propia a partir de resultados/govwa.json. N.° = cantidad de hallazgos de la regla.

Figura 2

Hallazgos de gosec sobre GoVWA por regla y severidad



Nota. Elaboración propia a partir de resultados/govwa.json.

Tabla 5

Riesgo y mitigación recomendada por regla

Regla	Riesgo	Mitigación recomendada
G701 y G201	El dato del usuario se une a la instrucción SQL y cambia su significado (CWE-89).	Consulta parametrizada y validación del tipo del identificador.
G203	template.HTML desactiva el escapado: un texto con etiquetas se ejecuta en el navegador (XSS, CWE-79).	Pasar texto plano a html/template; si se requiere HTML, sanitizarlo y añadir una política CSP.
G401 y G501	MD5 es muy rápido y no usa sal: las contraseñas de una base filtrada se recuperan con facilidad.	bcrypt o argon2id, y migrar los hashes existentes en el siguiente inicio de sesión.
G124	Cookies expuestas a robo por script (sin HttpOnly), al envío por HTTP (sin Secure) o a CSRF (sin SameSite).	Establecer Secure, HttpOnly y SameSite.
G112	Un cliente que envía las cabeceras muy despacio mantiene conexiones abiertas y agota el servidor (Slowloris).	Usar http.Server con ReadHeaderTimeout y los demás tiempos límite.
G706	Un texto con saltos de línea puede falsificar entradas del registro.	Registro estructurado (log/slog) y limpieza de los saltos de línea de los valores externos.
G104	Un error ignorado deja la aplicación en un estado no previsto.	Comprobar el error y responder de forma controlada.

Nota. Elaboración propia.

4.3 Hallazgos destacados

4.3.1 Inyección SQL (G701 y G201)

En vulnerability/sqli/function.go, la función UnsafeQueryGetData arma la consulta con fmt.Sprintf e inserta directamente el identificador recibido del usuario (uid); después la ejecuta con DB.Query. gosec lo reporta desde dos ángulos: la regla G201 detecta el SQL armado con formato de texto (líneas 37 a 40) y la regla G701, mediante análisis de flujo de datos, detecta que el dato del usuario llega hasta Query (línea 41). La salida de texto de gosec para el segundo hallazgo fue la siguiente (se acortaron la primera línea y la sangría para ajustarlas al ancho de la página):

```
[Gosec/objetivos/govwa/vulnerability/sqli/function.go:41] - G701 (CWE-89):
SQL injection via taint analysis: (*database/sql.DB).Query
(Confidence: HIGH, Severity: HIGH)
    40:     and u.id=%s`, uid) //here is the vulnerable query
  > 41:     rows, err := DB.Query(getProfileSql)
    42:     if err != nil {
```

El riesgo es que un atacante modifique el significado de la consulta para leer o alterar datos de otros usuarios. La mitigación es usar una consulta parametrizada, de modo que el dato viaje aparte de la instrucción SQL, y validar que el identificador sea numérico. El propio proyecto incluye junto a la función vulnerable una versión segura, SafeQueryGetData, que usa un marcador (?) y db.Prepare (líneas 66 a 75); gosec no la marcó, lo que sirve como control de que la regla no genera alarmas sobre el código corregido.

4.3.2 Contraseñas protegidas con MD5 (G401 y G501)

En user/user.go, el inicio de sesión calcula el MD5 de la contraseña (línea 88; la función Md5Sum ocupa las líneas 159 y 160) para compararlo con lo almacenado. gosec reporta la debilidad con dos reglas: G401, que detecta el uso de MD5 (CWE-328), y G501, que detecta la importación de crypto/md5 (CWE-327). El mismo patrón aparece en idor.go y csa.go, de modo que cada regla suma tres hallazgos y, entre las dos, seis. MD5 se calcula en nanosegundos y aquí

no hay sal: si la base de datos se filtra, las contraseñas comunes se recuperan casi de inmediato. La mitigación es usar bcrypt o argon2id y migrar los hashes existentes aprovechando el inicio de sesión: al validar una contraseña correcta con el hash antiguo, se guarda de nuevo con el algoritmo nuevo.

4.3.3 HTML sin escapar (G203)

En vulnerability/xss/xss.go, el término de búsqueda se toma de r.FormValue("term") (línea 39) y se convierte con template.HTML (líneas 62 y 63). Esa conversión le indica a Go que el texto ya es HTML seguro y desactiva el escapado automático de html/template. gosec reporta seis hallazgos G203 (cinco en xss.go y uno en util/template.go) con severidad media y confianza baja, porque no puede saber si el valor proviene del usuario. La revisión manual confirmó que en xss.go sí proviene del usuario, por lo que es un riesgo real de XSS (CWE-79): es el ejemplo de por qué la confianza importa, ya que lo que gosec califica con confianza baja debe verificarse antes de descartarlo. La aplicación intenta filtrar el texto con una función propia (removeScriptTag), pero filtrar con listas negras es frágil; lo correcto es pasar texto plano y dejar que html/template escape la salida.

4.4 Otros hallazgos

Las demás reglas son de severidad media o baja. La regla G124 señala dos cookies creadas sin atributos seguros (util/cookie.go, líneas 32 y 48). La regla G112 indica que el servidor de app.go (líneas 71 a 74) no configura ReadHeaderTimeout y es vulnerable a conexiones lentas. La regla G706 reporta once escrituras de valores externos en los registros (seis en user/session/session.go, tres en util/middleware/middleware.go, una en xss.go y una en function.go). La regla G104 reporta siete errores sin manejar (cuatro en idor.go, dos en util/template.go y uno en middleware.go).

5. Código propio: antes y después

5.1 Diseño del ejemplo

Para demostrar el ciclo completo de detectar, corregir y volver a escanear, se escribió un módulo pequeño en dos versiones: la versión vulnerable, de 73 líneas, que reúne los errores típicos de Go, y la versión corregida, de 126 líneas, con cada error resuelto y las mismas funciones con los mismos nombres. gosec reportó 17 hallazgos en la versión vulnerable, que corresponden a 15 reglas distintas, y ninguno activo en la versión corregida (Tabla 6). La Tabla 7 detalla la corrección aplicada a cada error.

Tabla 6

Resultados de gosec sobre el código propio antes y después de corregir

Medida	Vulnerable	Corregido
Líneas de código	73	126
Hallazgos activos	17	0
Severidad alta	6	0
Severidad media	9	0
Severidad baja	2	0
Reglas distintas	15	0
Hallazgos suprimidos con #nosec	0	2

Nota. Elaboración propia a partir de resultados/ejemplo_vulnerable.json y resultados/ejemplo_corregido.json.

Tabla 7

Errores del código propio y corrección aplicada

Error	Regla	Corrección aplicada
SQL armado con fmt.Sprintf	G201, G701	Consulta parametrizada con marcador (?)
Contraseña con MD5	G401, G501	bcrypt con sal y costo por defecto
Credencial escrita en el código	G101	Lectura desde la variable de entorno con os.Getenv
TLS sin verificar el certificado	G402	Verificación activa y TLS 1.2 como mínimo
Token con math/rand	G404	crypto/rand con 32 bytes
Cookie sin atributos seguros	G124	Secure, HttpOnly y SameSite estricto
Servidor sin tiempos límite	G114	http.Server con tiempos límite; solo en 127.0.0.1
Ruta escrita por el usuario	G304, G703	os.OpenRoot: no permite salir del directorio raíz
Comando con texto del usuario	G204, G702, G705	Lista de permitidos, sin shell ni salida devuelta
Errores sin manejar	G104	Comprobación del error y respuesta controlada

Nota. Elaboración propia a partir del código de ejemplo-propio/vulnerable y ejemplo-propio/corregido.

5.2 Falso positivo y supresión justificada

En la versión corregida, el comando ping se ejecuta solo si el destino pertenece a una lista cerrada de hosts permitidos y sin pasar por una shell. Aun así, gosec siguió reportando dos hallazgos (G204 y G702) porque no entiende esa validación: es un falso positivo. Se suprimió con un comentario que lleva además la justificación escrita (aquí partida en dos líneas por el ancho de la página):

```
// #nosec G204 G702 -- host validado contra una lista cerrada
//   y sin pasar por una shell
if err := exec.Command("ping", "-c", "1", host).Run(); err != nil {
```

Con la opción -track-suppressions, gosec no oculta la supresión: la registra junto con su justificación, de modo que queda auditable. El resumen de la ejecución reportó «Nosec: 1» (un comentario) y «Issues: 0», y el archivo JSON conserva los dos hallazgos suprimidos con su justificación. La supresión debe usarse con criterio: sin justificación ni revisión, un comentario #nosec puede ocultar un problema real.

6. Límites de la herramienta

Un análisis estático solo ve lo que sus reglas saben reconocer. Se documentaron los siguientes límites con evidencia obtenida de las ejecuciones.

6.1 Fallos de lógica: el caso del IDOR

GoVWA incluye en vulnerability/idor/idor.go un módulo vulnerable a referencia directa insegura a objetos (IDOR), es decir, que permite acceder a datos de otro usuario cambiando un identificador. En ese archivo gosec reportó seis hallazgos (cuatro errores sin manejar y el uso de MD5, con sus dos reglas), pero ninguno señala el IDOR. Decidir qué usuario debería ver qué dato es un asunto de lógica de negocio que ninguna regla de patrones puede conocer.

6.2 Secretos en archivos de configuración

La contraseña de la base de datos y la clave de sesión de GoVWA están en config/config.json. gosec solo analiza archivos .go, por lo que no las vio. Detectar secretos en archivos de configuración exige herramientas específicas de búsqueda de secretos.

6.3 Credenciales en el código: la regla G101

La regla G101 busca credenciales escritas en el código a partir del nombre de la variable y de la aleatoriedad (entropía) del valor. Para medir su alcance se escribió un experimento con cuatro variables que contienen credenciales (Tabla 8).

Tabla 8
Variables con credenciales marcadas por la regla G101

Variable	Valor	Configuración por defecto	Con ignore_entropy
claveLegible	Cadena legible de baja entropía	No marcada	No marcada
claveAleatoria	32 caracteres aleatorios	No marcada	No marcada
password	admin123	No marcada	Marcada
apiToken	32 caracteres aleatorios	Marcada	Marcada

Nota. Elaboración propia a partir de resultados/experimento_g101.txt. Los valores son de prueba.

Con la configuración por defecto solo se marcó una de las cuatro variables. La variable password con un valor trivial apareció únicamente al desactivar el filtro de entropía (ignore_entropy), y las variables cuyo nombre no contiene palabras como password, token o secret (aquí, clave…) nunca se marcaron, aunque contenían credenciales. La regla se apoya, por tanto, en palabras clave en inglés y en la entropía del valor, y no debe tomarse como garantía de que no hay credenciales en el código.

6.4 Falsos positivos y confianza baja

Los límites anteriores son falsos negativos. También hay falsos positivos, como el de los comandos validados con lista de permitidos (sección 5.2), y hallazgos de confianza baja que resultan reales, como el HTML sin escapar (sección 4.3.3). En ambos casos la decisión final

corresponde a una revisión humana: gosec acota dónde mirar, pero no reemplaza el criterio del revisor.

7. Experiencia de uso

La Tabla 9 resume la valoración del grupo según los criterios de la guía: instalación, facilidad de uso, resultados e interfaz, con la evidencia que la respalda.

Entre los aspectos favorables se destacan la rapidez del análisis, la claridad de cada hallazgo, la diversidad de formatos de salida y la posibilidad de filtrar por severidad y confianza, que facilita la priorización. En el informe HTML, además, el filtro deshabilita las categorías que no tienen hallazgos (por ejemplo, la confianza media en la Figura 1).

Entre los aspectos por mejorar están la versión «dev» que reporta el binario instalado desde el código fuente, el hecho de que solo analice código Go, la dependencia de la regla G101 de los nombres de variable, la necesidad de suprimir a mano los falsos positivos con comentarios #nosec y que el informe HTML requiere conexión a internet para cargar sus estilos.

Tabla 9

Valoración de la experiencia de uso de gosec

Criterio	Valoración	Evidencia
Instalación	Fácil	Un comando (go install) y unos 2 minutos de compilación. Requiere Go; no necesitó Docker, cuenta ni administrador.
Facilidad de uso	Muy fácil	gosec ./... es suficiente. El análisis completo de GoVWA (20 archivos) tardó unos 5 segundos.
Resultados	Claros, con falsos positivos y negativos	Cada hallazgo trae regla, CWE, severidad, confianza, línea y código. Hubo falsos positivos (G204, G702) y falsos negativos (IDOR, G101).
Interfaz	Terminal e informe HTML	El informe HTML ofrece filtros por severidad y confianza; también genera JSON, SARIF y JUnit para integración continua.
Integración continua	Buena	GitHub Action oficial; el código de salida distinto de cero cuando hay hallazgos permite hacer fallar un pipeline, y -no-fail lo evita cuando solo se quiere informar.

Nota. Elaboración propia a partir de las ejecuciones descritas en las secciones 2 a 6.

8. Recomendaciones

A partir de los resultados se proponen las siguientes prácticas para adoptar gosec en un proyecto Go. Primero, integrarlo en la integración continua con la Action oficial (securego, s. f.) y exportar los resultados en formato SARIF (OASIS, 2020), que facilita su consumo desde otras plataformas. Segundo, priorizar los hallazgos de severidad alta y confianza alta, para lo que sirve el filtro -severity=high -confidence=high (en GoVWA devolvió solo la inyección SQL), y revisar manualmente los de confianza baja.

Tercero, tratar los comentarios #nosec como una deuda: usarlos solo con justificación escrita y ejecutar gosec con -track-suppressions para que queden auditables. Cuarto, no confiar en la regla G101 como única defensa contra credenciales en el código ni en el análisis de archivos de configuración; conviene sumar una herramienta de búsqueda de secretos. Quinto, complementar el análisis estático con DAST, SCA y revisión manual, que cubren los fallos de lógica y la configuración. Por último, registrar la versión real de gosec con go version -m y fijar el commit del objetivo para que cada análisis sea repetible.

Conclusiones

gosec se instaló con un solo comando, sin Docker ni permisos de administrador, y analizó una aplicación de 20 archivos en unos 5 segundos. Reportó 35 hallazgos sobre GoVWA, entre ellos la única inyección SQL de severidad alta, y sus resultados son claros porque cada uno indica regla, CWE, severidad, confianza y código.

El ciclo con código propio mostró el valor práctico de la herramienta: los 17 hallazgos de la versión vulnerable pasaron a 0 activos tras aplicar las mitigaciones, con dos falsos positivos suprimidos de forma justificada y auditable. A la vez, la evidencia documentó sus límites: no

detectó el IDOR, no revisó los secretos de config.json y la regla G101 depende del nombre de la variable y de la entropía del valor.

En conclusión, gosec es una herramienta recomendable como primera barrera automática en proyectos Go, especialmente dentro de la integración continua, siempre que sus resultados se interpreten con la severidad y la confianza, se revisen a mano los casos dudosos y se complemente con pruebas dinámicas, análisis de dependencias y revisión del diseño.

Referencias

0c34. (s. f.). *GoVWA: Go Vulnerable Web Application* [Software]. GitHub. https://github.com/0c34/govwa

MITRE. (s. f.). *Common Weakness Enumeration (CWE)*. https://cwe.mitre.org/

OASIS. (2020). *Static Analysis Results Interchange Format (SARIF) Version 2.1.0* (OASIS Standard). https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html

OWASP Foundation. (s. f.). *Static code analysis*. https://owasp.org/www-community/controls/Static_Code_Analysis

securego. (s. f.). *gosec: Go security checker* [Software]. GitHub. https://github.com/securego/gosec

Anexo A. Reproducción de los resultados

Los análisis pueden repetirse desde la carpeta Gosec del repositorio del trabajo. Se requieren Go 1.25 o superior, Git, Python y un intérprete de Bash. Los siguientes comandos instalan gosec, escanean GoVWA (el script clona el commit fijado), escanean el código propio antes y después de corregir, repiten el experimento de la regla G101 y resumen los resultados:

```
go install github.com/securego/gosec/v2/cmd/gosec@latest
bash scripts/escanear.sh
bash scripts/escanear_ejemplo.sh
bash scripts/experimento_g101.sh
py scripts/analizar.py
```

El script analizar.py resume los hallazgos por severidad, confianza, regla y archivo; con la opción --csv escribe además el archivo resultados/hallazgos.csv. El detalle de la estructura de la carpeta se encuentra en el archivo README.md del trabajo.